

Reliability tradeoffs

A reliable workload consistently meets its defined reliability objectives. It should reach established resiliency targets, ideally by circumventing events that affect reliability. Realistically, however, a workload must tolerate and control the impact of such events and maintain operations at a predetermined level during active malfunction. Even during a disaster, a reliable workload must recover to a specific state within a given period of time, both of which are agreed upon among the stakeholders. An incident response plan that enables you to achieve rapid detection and recovery is vital.

When you design a workload, consider how decisions based on the [Reliability design principles](#) and the recommendations in the [Design review checklist for Reliability](#) might influence the goals and optimizations of other pillars. Some reliability decisions benefit one pillar but constitute tradeoffs for another. This article describes example tradeoffs that a workload team might encounter when designing workload architecture and operations for reliability.

Reliability tradeoffs with Security



Tradeoff: Increased workload surface area. The Security pillar prioritizes a reduced and contained surface area to minimize attack vectors and reduce the management of security controls.

- Reliability is often obtained through replication at the component level, the data level, or even the geographic level. Replicas, by design, increase the workload's surface area. From a security perspective, a reduced and contained surface area is preferred to minimize potential attack vectors and streamline the management of security controls.
- Similarly, disaster recovery solutions, like backups, increase a workload's surface area. However, they're often isolated from the workload's runtime. These solutions require the implementation of additional security controls, which might be specific to the disaster recovery approach.
- Reliability goals can require additional architectural components, like a message bus added to make requests resilient through decoupling. Each new component must be secured, possibly in ways that aren't already used in the system. The supporting code and libraries also expand the application's surface area.



Tradeoff: Security control bypass. The Security pillar recommends that all controls remain active in both normal and stressed systems.

- When a workload is experiencing a reliability event that's being addressed under active incident response, urgency might create pressure for workload teams to bypass security controls that are optimized for routine access.
- Troubleshooting activities can cause the team to temporarily disable security protocols, leaving an already stressed system potentially exposed to additional security risks. There's also a risk that the security protocols won't be reestablished promptly.
- Granular implementations of security controls, like custom role-based access control assignments or narrow firewall rules, introduce configuration complexity and sensitivity, increasing the chance for misconfiguration. Mitigating this potential reliability impact by using broad rules erodes all three Zero Trust architecture principles.



Tradeoff: Old software versions. The Security pillar encourages a "get current, stay current" approach to vendor security patches.

- Applying security patches or software updates can potentially disrupt the target component, causing unavailability during the software change. Delaying or avoiding patching might avoid the potential reliability risks, but it leaves the system unprotected against evolving threats.
- The preceding consideration also applies to the workload's code. For example, it applies to application code that uses old libraries and containers that use old base images. If updating and deploying application code is viewed as an unmitigated reliability risk, the application is exposed to additional security risks over time.

Reliability tradeoffs with Cost Optimization



Tradeoff: Increased implementation redundancy or waste. A cost-optimized workload minimizes underutilized resources and avoids over-provisioning resources.

- Replication is a key strategy for reliability. Specifically, the strategy is to have enough replication to handle a given number of concurrent node failures. The tolerance for more concurrent node failures requires a higher replica count, which leads to increased costs.

- Over-provisioning is another technique for absorbing unexpected load on a system, such as during a failover event, that could otherwise lead to a reliability issue. Any excess capacity that's not utilized is considered wasteful.
- If a workload uses a disaster recovery solution that excessively satisfies the workload's recovery point and time objectives, the excess leads to higher costs because of waste.
- Workload deployments themselves are a potential source for reliability impact, and that impact is often mitigated by redundancy at deployment time via a deployment strategy like blue/green. This transient duplication of resources during safe deployment typically increases the overall cost of the workload during those periods. Costs increase with frequency of deployments.



Tradeoff: Increased investment in operations that aren't aligned with functional requirements. One approach to cost optimization is evaluating the value that's provided by any deployed solution.

- To achieve reliability, a system requires observability. Monitoring systems require observability data transfer and collection. As monitoring capabilities increase, the frequency and volume of data increase, leading to additional costs.
- Reliability affordances in workloads necessitate testing and drills. Designing and running tests takes time and potentially specialized tooling, which incurs costs.
- Workloads with high reliability targets often have a rapid response process that requires technical team members to be part of a formal on-call rotation. This process incurs additional personnel costs and lost opportunity costs because of attention that could be directed elsewhere. It also incurs potential tooling costs for management of the process.
- Support contracts with technology providers are a key component of a reliable workload. Support contracts that aren't utilized because the level of support is over-provisioned incur waste.

Reliability tradeoffs with Operational Excellence



Tradeoff: Increased operational complexity. Operational Excellence, like Reliability itself, prioritizes simplicity.

- Reliability usually increases the complexity of a workload. As the complexity of a workload increases, the operational elements of the workload can also increase to support the added components and processes in terms of deployment coordination and configuration surface area.
- Having a comprehensive monitoring strategy for a workload is a key part of operational excellence. Introducing additional components into an architecture to implement reliability design patterns results in more data sources to manage, increasing the complexity of implementing distributed tracing and observability.
- Using multiple regions to overcome single region resource capacity constraints and/or implement an active/active architecture increases the complexity of the workload's operational management. This complexity is introduced by the need to manage multiple regions and the need to manage the data replication between them.



Tradeoff: Increased effort to generate team knowledge and awareness. The Operational Excellence pillar recommends keeping and maintaining a documentation repository for procedures and topologies.

- As a workload becomes more robust through the addition of reliability components and patterns, it takes more time to maintain operational procedures and artifact documentation.
- Training becomes more complex as the number of components in the workload increases. This complexity affects onboarding time. The complexity also increases the knowledge needed to track product roadmaps and the latest service-level guidance.

Reliability tradeoffs with Performance Efficiency



Tradeoff: Increased latency. Performance Efficiency requires a system to achieve performance targets for user and data flows.

- Reliability patterns often incorporate data replication to survive replica malfunction. Replication introduces additional latency for reliable data-write operations, which consumes a part of the performance budget for a specific user or data flow.
- Reliability sometimes employs various forms of resource balancing to distribute or redistribute load to healthy replicas. A dedicated component that's used for balancing usually affects the performance of the request or process that's being balanced.

- Distributing components across geographical boundaries or availability zones to survive a scoped impact introduces network latency in the communication between components that span those availability boundaries.
- Extensive processes are used to observe the health of a workload. Although monitoring is critical for reliability, instrumentation can affect system performance. As observability increases, performance might decrease.



Tradeoff: Increased over-provisioning. The Performance Efficiency pillar discourages over-provisioning, instead recommending the use of just enough resources to satisfy demand.

- Automatic scaling operations aren't instantaneous and therefore can't reliably handle a sudden and dramatic spike in demand that can't be shaped or smoothed. Therefore, over-provisioning via either larger instances or more instances is a critical reliability tactic to account for the lag between demand signal and supply creation to help absorb bursts. Unused capacity counters the goals of performance efficiency.
- Sometimes a component can't be scaled in reaction to demand, and that demand isn't fully predictable. Using large instances to cover the worst case leads to over-provisioning waste in situations that are outside that use case.

Related links

Explore the tradeoffs for the other pillars:

- [Security tradeoffs](#)
- [Cost Optimization tradeoffs](#)
- [Operational Excellence tradeoffs](#)
- [Performance Efficiency tradeoffs](#)

Last updated on 04/27/2026